

Case Study 5: Smart Transportation and Traffic Analytics System using Apache Spark

A metropolitan smart city authority collects large-scale transportation data from GPS devices, traffic cameras, toll plazas, public transportation systems, and IoT traffic sensors. The authority wants to build a **Smart Transportation Analytics Platform** using Apache Spark to analyze traffic congestion, optimize routes, predict travel times, and improve urban mobility. The final application should be developed and deployed using DevOps methodologies.

As a Big Data Engineer, design, implement, and deploy the solution using Apache Spark and PySpark.

Q1. Spark Initialization and Data Loading (2 Marks)

Initialize **Spark Session** and **Spark Context**. Load transportation and traffic datasets and display the schema and sample records.

Q2. RDD Implementation (3 Marks)

Create **RDDs** from transportation datasets and perform transformation and action operations to analyze traffic and travel data.

Q3. Key-Value Operations and Persistence (2 Marks)

Implement **key-value pair operations**, perform **shuffle operations**, and demonstrate **RDD persistence** for traffic analytics.

Q4. Spark DataFrame Operations (3 Marks)

Convert transportation datasets into **Spark DataFrames** and perform selection, filtering, grouping, joins, and aggregation operations.

Q5. Exploratory Data Analysis and Spark SQL (5 Marks)

Perform EDA and execute Spark SQL queries to:

- Identify peak traffic hours.
- Analyze route-wise traffic density.
- Determine average travel time by location.
- Identify congestion hotspots.
- Generate daily and monthly traffic trend reports.

Q6. ETL Pipeline Development (3 Marks)

Design and implement an ETL pipeline to extract data from traffic sensors, GPS devices, and transportation databases, transform the data, and load it into a centralized analytics repository.

Q7. Machine Learning/Deep Learning Implementation (2 Marks)

Implement a Machine Learning/Deep Learning model to predict traffic congestion, travel time, or accident probability and evaluate its performance.

Additional Implementation Instructions

A. Version Control using Git and GitHub

- Create a **GitHub repository** for the project.
- Upload all source code, datasets (or dataset links), notebooks, and documentation.
- Maintain proper folder structure and commit history.
- Include a **README.md** file containing project description, installation steps, execution procedure, and output screenshots.

B. Containerization using Docker

- Create a **Dockerfile** for the Spark application.
- Build and test the Docker image.

C. Deployment using Kubernetes

- Create Kubernetes deployment and service YAML files.
- Deploy the application on a Kubernetes cluster (Minikube/Kubernetes/OpenShift).
- Verify successful deployment.

D. DevOps Pipeline Implementation

- Configure a basic CI/CD pipeline using GitHub Actions/Jenkins/GitLab CI.
- Automate build, testing, Docker image creation, and deployment.

E. Project Submission Requirements

Submit:

- Source code
- GitHub repository link
- Dockerfile
- Kubernetes YAML files
- CI/CD pipeline configuration
- Execution screenshots
- EDA and Spark SQL outputs
- Model evaluation report
- Final project documentation